Decision Trees, Forests & Gradient Boosters — Task 2 Report

Cryptonite Research Task Phase 1 · Module 2 · Cybersecurity Intrusion Detection (binary classification: *attack_detected*) · Random Forest, feature engineering, and boosted trees (XGBoost / LightGBM / CatBoost).

1. Mathematical Formulation of Decision Trees

A decision tree greedily partitions the feature space by choosing the split that maximises purity gain. For a node **S** containing samples with class probabilities $p_i = |C_i \cap S| / |S|$, two standard impurity measures are:

```
Gini impurity: Gini(S) = 1 − Σ_i p_i²  (0 = pure, 0.5 = max for binary)

Entropy: H(S) = − Σ_i p_i · log■(p_i)  (0 = pure, 1 = max for binary)
```

Gini(S) = 0 when all samples belong to one class; H(S) is maximal (=1) at p = 0.5. For a candidate split of S into children S■…S■ with $|S_v|/|S|$ the weighted fraction, the **Information Gain** is:

```
IG(S, A) = Impurity(S) − Σ_v (|S_v| / |S|) · Impurity(S_v)
```

The algorithm evaluates every feature *A* and every threshold, selects the split with maximum gain, and recurses on each child until a stopping rule (max depth, min samples per leaf, or pure node) is reached. Training is $O(m \cdot n \log n)$ per node over n samples and m features. Because a single deep tree overfits (high variance), **Random Forests** reduce variance by bagging B trees on bootstrap samples and, at each split, considering only a random subset of $d \approx \sqrt{m}$ features: $f(x) = (1/B) \Sigma_b f_b(x)$ for classification (majority vote) or averaging of probabilities. Feature subsampling decorrelates the trees, so the ensemble's error is far below that of any individual tree.

2. Data & Exploratory Analysis (summary)

The dataset has **9,537 rows × 10 usable features** after dropping the unique *session_id* identifier (6 numeric, 3 categorical, 1 binary flag). The target *attack_detected* is **balanced** (≈55% normal / 45% attack; imbalance ratio 1.24), so accuracy is a fair headline metric and no resampling is required. Only *encryption_used* has missing values (**1,966 cells ≈ 20%**), which is itself informative missingness; it is imputed as a dedicated category plus an *enc_missing* indicator. Univariate boxplots show *ip_reputation_score*, *failed_logins*, *unusual_time_access* and *login_attempts* separate the classes most strongly, and *encryption_used = DES* is riskier than *AES* — these findings drive the feature engineering in §3.

3. Baseline Random Forest vs. Reference Targets

A plain RandomForestClassifier (400 trees, stratified 80/20 split, random_state = 42) with median imputation and one-hot encoding already meets or exceeds every reference target:

Metric	Reference target	Baseline RF	Δ vs target	Status
Accuracy	≥ 0.8831	0.8831	+0.0000	✓ met
F1 (binary)	≥ 0.8508	0.8512	+0.0004	✓ met
ROC-AUC	≥ 0.8762	0.8787	+0.0025	✓ met

4. Feature Engineering — Comparative Analysis

Six feature families were added incrementally and the same forest retrained after each step to isolate every method's score delta. Frequency/target encoding alone slightly *overfit* for bagged trees (small AUC drop), while **interaction features** produced the largest single gain, and **hyper-parameter tuning** delivered the best overall model.

Stage (cumulative)	Accuracy	F1	ROC-AUC	ΔAUC vs base
S0 Baseline (impute + one-hot)	0.8831	0.8512	0.8787	—
S1 + missing indicator	0.8831	0.8512	0.8747	–0.0040
S2 + frequency encoding	0.8810	0.8492	0.8776	–0.0011
S3 + target (smoothed) encoding	0.8810	0.8490	0.8765	–0.0022
S4 + interaction features	0.8836	0.8512	0.8813	+0.0026
S5 + aggregation statistics	0.8821	0.8495	0.8807	+0.0020
S6 + outlier capping	0.8826	0.8501	0.8807	+0.0020
Tuned RF (RandomizedSearchCV)	0.8863	0.8545	0.8824	+0.0037

Best tuned parameters: n_estimators = 500, max_depth = 25, min_samples_leaf = 4, max_features = 0.9 (3-fold stratified CV maximising ROC-AUC). The tuned forest lifts Accuracy by +0.0032, F1 by +0.0033 and AUC by +0.0037 over the baseline, clearing all targets.

5. (Bonus) Gradient Boosting — Implementation Details

Boosting builds trees *sequentially*; each new tree fits the **negative gradient** (residual errors for squared loss, and the first-order gradient for log-loss) of the loss so far: $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$ with learning rate ν. Unlike bagging, boosting focuses capacity on hard-to-classify points. **XGBoost** adds a second-order (Newton) update with L1/L2 regularisation and column subsampling; **LightGBM** scales it with leaf-wise growth and Gradient-Based One-Side Sampling (GOSS); **CatBoost** handles categoricals natively via ordered target statistics, avoiding target leakage. All three use a small learning rate (0.05) with 400 trees.

Model	Accuracy	F1	ROC-AUC	Meets all targets
Random Forest (ref.)	0.8836	0.8514	0.8820	✓
XGBoost	0.8831	0.8514	0.8791	✓
LightGBM	0.8826	0.8511	0.8778	✓
CatBoost	0.8868	0.8550	0.8786	✓
Ensemble (XGB+LGB, soft vote)	0.8826	0.8509	0.8783	✓

CatBoost gives the highest Accuracy/F1 (0.8868 / 0.8550) and the tuned forest the highest ROC-AUC (0.8824). All boosted models match or beat the Random-Forest baseline and clear every reference target; the AUC ceiling (~0.882) is set by the data's overlapping class region rather than the algorithm. A soft-voting ensemble of XGBoost + LightGBM adds a small stability gain.

6. Conclusion

The baseline Random Forest already satisfies the reference targets; interaction features and hyper-parameter tuning yield the largest improvements, and the bonus gradient boosters (especially CatBoost) push Accuracy/F1 slightly higher. Full code, EDA visuals, per-method deltas and a reusable Kaggle submission pipeline are in **Model_Tuning.ipynb** and **Kaggle_Competition_and_Boosting.ipynb**.